A Trimester – 1 Class Project Report on

SMART CARE – HOSPITAL MANAGEMENT SYSTEM

Submitted in the partial fulfillment of the requirements for

26CS5101E – Computational Thinking and Programming

Master of Technology In

Artificial Intelligence & Data Science

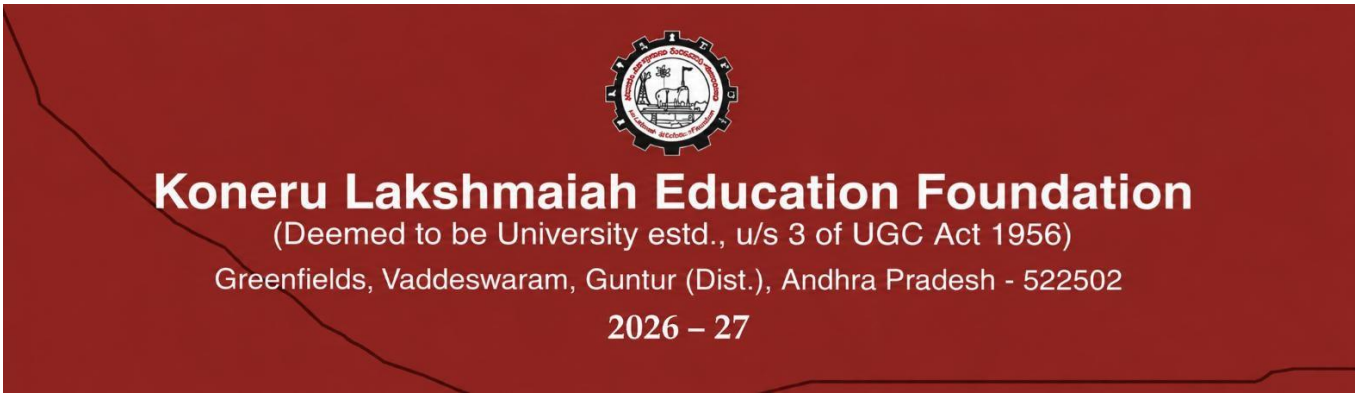
By

Student ID	Student Name
2601150059	KOLLAVENKATESH

under the supervision of

Dr. G. Syam Prasad

Professor



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



CERTIFICATE

This is to certify that the Trimester – 1 Class Project Report entitled "**Smart Care – Hospital Management System**" is being submitted by KOLLA VENKATESH (Reg. no. 2601150059) submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University is a record of Bonafide work carried out under our guidance and supervision during the academic year 2026 – 2027.

Dr. G. Syam Prasad

Project Supervisor

Signature

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



DECLARATION

The Trimester-1 Class Project Report entitled "**Smart Care – Hospital Management System**" is a record of Bonafide work of KOLLA VENKATESH (Reg. no. 2601150059) submitted in partial fulfillment the requirements for the course 26CS5101E – Computational Thinking and Programming, Master of Technology in Artificial Intelligence and Data Science to the K L University.

P;

KOLLA VENKATESH - 2601150059

ACKNOWLEDGEMENT

It is great pleasure for me to express my gratitude to our honorable President **Sri. Koneru Satyanarayana**, for giving the opportunity and platform with facilities in accomplishing the trimester - 1 class project report.

I express sincere gratitude to HOD of AI&DS Dept. **Dr. M. Venkata Naresh** for his leadership and constant motivation provided in successful completion of our academic semester. I record it as my privilege to deeply thank for providing us the efficient faculty and facilities to make our ideas into reality.

I express my sincere thanks to our project supervisor **Dr. G. Syam Prasad** for his novel association of ideas, encouragement, appreciation, and intellectual zeal which motivated us to venture this project successfully.

Finally, it is pleased to acknowledge the indebtedness to all those who devoted themselves directly or indirectly to make this project report success.

ABSTRACT

Smart Care is a desktop hospital management mini-project developed using Python, Tkinter, and SQLite. It allows front-desk staff to register patients and doctors, schedule appointments, mark appointments as completed, create bills, and record payment status from a single tabbed window. Data is stored locally in a SQLite database with four related tables: patients, doctors, appointments, and bills. The database is created automatically on first launch, so no database server or third-party package is required. The project applies computational thinking through problem decomposition, abstraction, pattern recognition, algorithm design, data structures, and complexity analysis. Required-field checks, an age range check, date-time format validation, non-negative bill amounts, parameterized SQL queries, and deletion rules that protect records with linked appointments or bills support data integrity. The source code was checked for successful compilation and the database layer was exercised with sample data; the project does not include an automated unit-test suite. Smart Care is an educational project and not a production clinical system.

INDEX

S. No	TITLE	PAGE NO
1	Introduction	
2	Problem Statement	
3	System Architecture and Technologies	
4	Problem Solving & Computational Thinking	
5	Programming & Data Structures	
6	Software Design & Engineering	
7	Functional Programming & Concurrency	
8	Software Quality & DevOps Practices	
9	AI-Assisted Software Engineering	
10	Algorithms, Complexity and Performance	
11	Implementation and User Workflow	
12	Testing, Results and Limitations	
13	Conclusion and Future Enhancements	
14	References	

1. INTRODUCTION

Smart Care is a desktop application that digitizes the core routine of a small hospital or clinic: registering patients and doctors, scheduling appointments, and billing. It is built with Python's Tkinter library for the graphical interface and SQLite for persistent storage, so it runs on a single computer without a web server or any installation beyond Python itself.

The project is relevant to Computational Thinking and Programming because it converts a practical administrative problem into smaller, testable tasks: data modelling, input validation, record creation, relational lookup, status updates, and display refresh. Each task maps to a clear function or class method in main.py.

1.1 Objectives:

- Register patients (name, age, gender, phone, address) and list them in a table.
- Register doctors with their department and contact number.
- Schedule appointments that link a patient and a doctor, and mark them as completed.
- Create bills for patients and update the payment status from Unpaid to Paid.
- Validate user input and protect records that have linked history from being deleted.
- Store all data locally in an automatically created SQLite database.

2. PROBLEM STATEMENT

Small clinics often record patients, doctor schedules, and payments in paper registers or scattered spreadsheets. This makes it slow to find a patient's history, easy to enter inconsistent data (for example, an impossible age or a badly written date), and difficult to see which appointments are still pending or which bills remain unpaid. A useful solution should keep related records connected, validate data at the point of entry, avoid deleting records that other records depend on, and present everything in a simple interface that non-technical staff can use.

21 Proposed Solution:

Smart Care provides a four-tab desktop window: Patients, Doctors, Appointments, and Billing. Each tab has an entry form at the top and a table of saved records below it. The appointment and billing forms choose patients and doctors from drop-down lists built from the database, so records are linked by identifier instead of being retyped. Status buttons mark appointments as Completed and bills as Paid.

22 Scope:

The system covers patient and doctor registration, appointment scheduling and completion, bill creation and payment marking, and deletion of patients and doctors that have no linked records. It is a single-user, single-computer application. It does not include login or user roles, record editing, appointment cancellation, doctor availability checking, medical records, prescriptions, reports, or insurance handling. The README states that only fictional data should be used.

23 Expected Outcomes:

The expected outcome is a working desktop application in which a user can complete the registration-to-billing workflow in one place, with validated inputs and consistently linked data. The supplied project consists of one source file, main.py (about 345 lines), and a README. Usability on another computer depends on Tkinter being installed together with Python.

3. SYSTEM ARCHITECTURE AND TECHNOLOGIES

The application follows a simple single-process, layered design. The Tkinter interface sends user actions to event-handler methods of the SmartCareApp class, which validate the input and run SQL statements against a local SQLite database file. After each change, the interface reloads its tables from the database.

31 Architecture Flow:

• Tkinter interface: a ttk.Notebook with four tabs (Patients, Doctors, Appointments, Billing); each tab has a form (LabelFrame) and a Treeview table with a scrollbar.

• Application class (SmartCareApp): builds the window, holds the form variables, validates input, and handles button events such as add_patient, add_appointment, and pay_bill.

• Data-access helpers: connect_db() opens a connection to smart_care_hospital.db and initialize_db() creates the four tables if they do not already exist.

• SQLite database (smart_care_hospital.db): stores patients, doctors, appointments, and bills in the folder from which the program is run.

• Refresh cycle: refresh_all() re-reads all tables, repopulates the four Treeview tables, and updates the patient and doctor drop-down lists.

32 Technology Stack:

• Python 3.10+: implements all application logic.

• Tkinter and ttk: provide the window, tabs, forms, drop-down lists, tables, and message boxes (included with the standard Python installer on Windows).

• SQLite (sqlite3 module): provides local relational storage without a separate database server.

• SQL: parameterized INSERT, UPDATE, DELETE, and JOIN queries create, change, and combine records.

• datetime module: validates the appointment date-time text and records bill creation time.

• No third-party packages: the README lists none, so no pip installation is needed.

33 Main Modules:

The project has a single source file, main.py. It contains the constants APP_TITLE and DB_NAME, the functions connect_db() and initialize_db(), and the class SmartCareApp. The class is organized into builder methods (build_patients, build_doctors, build_appointments, build_bills), action methods (add, delete, complete, and pay operations), and refresh_all(). A helper method, make_tree(), creates the scrollable table used on every tab. README.md provides setup and demonstration steps.

4. PROBLEM SOLVING & COMPUTATIONAL THINKING

The project applies the computational thinking process by breaking the broad problem of clinic front-desk administration into small tasks, each with defined inputs, a rule, and an output.

41 Problem Decomposition:

The problem is divided into four domains (patients, doctors, appointments, and billing). Each domain is split into the same repeating tasks: collect input, validate, store, display, and update status. Dedicated methods exist for these tasks, such as add_patient, delete_doctor, complete_appointment, and pay_bill, and a shared refresh_all() redraws the display.

42 Abstraction:

Real-world people and events are reduced to a few essential fields. A patient is represented by an id, name, age, gender, phone, and address; a doctor by an id, name, department, and phone; an appointment by a patient, a doctor, a date-time, a reason, and a status; and a bill by a patient, an amount, a description, a payment status, and a creation time. Details such as medical history, doctor timetables, or insurance are deliberately left out.

43 Pattern Recognition:

All four tabs share one pattern: a form, a validation step, a parameterized INSERT, a table refresh, and a confirmation message. Appointments and bills also share a status workflow in which a record starts in an initial state (Scheduled or Unpaid) and moves to a final state (Completed or Paid). Recognizing these patterns allowed one make_tree() helper and one refresh_all() routine to serve every tab.

44 Algorithm Selection:

Input validation uses simple rule checks: required-field tests, an integer range check for age (0 to 130), datetime.strptime for the date-time format, and a non-negative number check for bill amounts. Relational lookups use SQL JOINs on primary keys so that patient and doctor names appear in the appointment and billing tables. Deletion follows a count-then-decide rule: a record is deleted only when no appointment or bill refers to it. Drop-down choices are stored as "id - name" text, and the id is recovered by splitting at the first " - ". The project does not require dynamic programming, backtracking, or branch-and-bound because it contains no optimization problem.

45 Complexity Analysis:

Let P, D, A, and B be the numbers of patients, doctors, appointments, and bills. Inserting a record or updating a status by primary key is approximately O(log n) in SQLite's B-tree storage. The refresh_all() routine reads and redraws every row, so each refresh is O(P + D + A + B), plus the cost of sorting the date-ordered appointment list and the name-ordered drop-down lists. The delete checks count linked appointments and bills; because the foreign-key columns are not separately indexed in the schema, these counts scan the related tables and are O(A + B) in the worst case. For a small clinic this cost is negligible, but a full refresh after every action would become noticeable with very large tables.

46 Data Integrity as a Design Constraint:

Hospital records should not be silently lost. The application therefore refuses to delete a patient who has appointments or bills, and a doctor who has appointments. All SQL statements use parameter placeholders instead of string concatenation. NOT NULL constraints in the schema and the validation checks in the interface protect required fields. The README notes that a real deployment would also need authentication, role-based permissions, audit logs, secure backups, and privacy controls.

5. PROGRAMMING & DATA STRUCTURES

Python is used for the interface, validation logic, and database access. The implementation relies on the standard library (tkinter, sqlite3, and datetime) and built-in data structures, which keeps the solution simple to run and maintain.

51 Data Structures:

• Relational tables: patients, doctors, appointments, and bills hold the permanent data. Appointments and bills reference patients (and doctors, for appointments) through FOREIGN KEY declarations.

• Tuples and lists: each SQL result row is a tuple, and the full result set is a list used to fill the Treeview tables and drop-down lists.

• Tkinter variables (StringVar): hold the current contents of form fields and drop-down selections.

• Delimited strings: drop-down entries such as "3 - Dr. Rao (Cardiology)" carry the record id, which is extracted with split(" - ", 1).

• Treeview widgets: display each table as rows of values and provide the selected row's id for the delete, complete, and pay actions.

52 Database Design:

The database has four tables: patients (id, name, age, gender, phone, address); doctors (id, name, department, phone); appointments (id, patient_id, doctor_id, appointment_date, reason, status with default "Scheduled"); and bills (id, patient_id, amount, description, payment_status with default "Unpaid", created_at). Each id is an auto-incrementing primary key. Appointment dates are stored as text in the format YYYY-MM-DD HH:MM, which sorts in chronological order when compared as text.

53 Input Validation:

Patient registration requires a name and an age, and the age must be an integer from 0 to 130. Doctor registration requires a name and a department. An appointment requires a patient, a doctor, and a date-time matching YYYY-MM-DD HH:MM. A bill requires a patient and an amount that is a number and not negative. Phone and address fields are optional and are not format-checked. A failed check shows a warning or error message box and nothing is saved.

5.4 Efficient Use of the Database:

Queries use JOINs to fetch patient and doctor names in one step instead of looking them up separately for every row. A short-lived connection is opened for each user action inside a with block, which commits the transaction when the block finishes successfully (the block commits or rolls back but does not itself close the connection). This is simple and adequate for a single-user desktop application.

6. SOFTWARE DESIGN & ENGINEERING

The application is organized into constants, database helper functions, and a single interface class whose methods are grouped by responsibility. Although all code is in one file, the grouping reduces coupling between screen construction, event handling, and data access.

6.1 Separation of Responsibilities:

Builder methods only construct widgets; action methods validate input and write to the database; refresh_all() reads from the database and updates the widgets. initialize_db() is the only place that defines the schema. This separation makes it clear where to look when changing a form, a rule, or a table.

6.2 Reusability and Maintainability:

The make_tree() method is reused for all four tables, and refresh_all() is reused after every change. Constants such as APP_TITLE and DB_NAME are defined once at the top of the file. The code is not split into several modules, and the available source does not show type hints or a formal design pattern, so these are not claimed. A natural improvement would be to move database operations into a separate module.

6.3 Error Handling and Usability:

User mistakes are handled with message boxes: warnings for missing details and errors for an invalid age, date, or amount. Attempts to delete a patient or doctor that has linked records are blocked with an explanatory message. The interface does not catch database exceptions such as an unwritable or locked database file, so an unexpected failure would appear as an error in the console rather than as a friendly message.

7. FUNCTIONAL PROGRAMMING & CONCURRENCY

Smart Care is an event-driven, single-threaded program. After the window is built, Tkinter's mainloop() waits for user actions and calls the matching handler method. Each handler follows a short pipeline: read the form values, validate them, write to the database, refresh the display, and notify the user.

7.1 Functional Decomposition:

Small single-purpose methods such as clear_patient_form, complete_appointment, and pay_bill keep the flow easy to follow. These methods read and change form variables, widgets, and the database, so they are side-effecting and should not be described as pure functions. The list comprehensions in refresh_all() that turn query rows into "id - name" option strings are simple data transformations.

7.2 Concurrency Assessment:

The source does not use threading, multiprocessing, or asyncio, so concurrency is not claimed as an implemented feature. All database calls run on the interface thread. With small local data this causes no noticeable delay, but a long-running query would freeze the window until it finished. If two copies of the program used the same database file, SQLite's own file locking would apply, but the application contains no logic of its own to handle that situation.

7.3 GIL and Workload Considerations:

Python's Global Interpreter Lock limits parallel execution of Python bytecode in threads, but Smart Care performs short database operations and spends most of its time waiting for user input, so the GIL is not a practical constraint here. If background work such as report generation were added, it should run outside the interface thread and pass results back to Tkinter safely, because Tkinter widgets should be updated from the main thread.

8. SOFTWARE QUALITY & DEVOPS PRACTICES

Software quality in the supplied project is supported by input validation, database constraints, protective deletion rules, and a README with setup and demonstration steps. The project does not include an automated test suite.

81 Verification Performed for This Report:

• Syntax check: main.py compiled successfully with Python 3.12.

• Schema creation: initialize_db() was run twice on a new database; the four tables were created and the second run caused no error.

• Sample data and queries: a doctor, patient, appointment, and bill were inserted; the default statuses were Scheduled and Unpaid; the JOIN query used by the appointment table returned the expected row.

• Constraint and safety checks: a NOT NULL violation was rejected, and SQL-injection-style text was stored as plain text, leaving the tables intact. The graphical interface itself was not launched because Tkinter was unavailable in the checking environment.

82 Additional Recommended Tests:

• Invalid inputs: empty name, non-numeric or out-of-range age (for example -1 and 131), negative or non-numeric amount, and wrong date format.

• Deletion rules: patients and doctors with and without linked appointments or bills.

• Status updates: marking an appointment Completed and a bill Paid, including clicking the button with nothing selected.

• Conflicting data: the same doctor booked twice at the same time, and duplicate patient names.

• Persistence: closing and reopening the application keeps all records.

• Automated unit tests (unittest) that run the database functions against a temporary database file.

83 Execution and Dependencies:

The project requires Python 3.10 or newer with Tkinter, which is included with the standard Python installer on Windows. No packages need to be installed. From the extracted smart_care_hospital_management folder, run python main.py. A window titled "Smart Care - Hospital Management System" opens, and the database file smart_care_hospital.db is created automatically in the folder from which the command is run.

The supplied project does not include a configured CI/CD pipeline, containerization workflow, code-coverage report, linting setup, or static type-checking pipeline, so these are not reported as completed DevOps features. The README is the only supporting documentation.

84 Local Security:

Smart Care stores personal and billing data in an unencrypted local SQLite file and has no login, so anyone with access to the computer or the file can read it. SQL statements use parameter placeholders, which avoids SQL injection through form fields. As the README states, a real deployment would need authentication, role-based permissions, audit logs, secure backups, and privacy controls. Only fictional data should be used.

9. AI-ASSISTED SOFTWARE ENGINEERING

AI-assisted software engineering is one of the areas listed in the Project Defence requirements. For this project, AI tools can support requirement clarification, code explanation, test-case design, debugging, and documentation. However, the use of any particular AI tool or generated code must be described truthfully based on the development process actually followed.

9.1 Specification-First Development:

A specification-first workflow starts by defining the required behavior: register patients and doctors, schedule appointments, create and settle bills, validate inputs, and keep linked records safe. These statements act as acceptance criteria against which the implemented features and checks can be compared.

9.2 Review and Validation:

AI-generated suggestions should not be accepted without review. In a hospital-style application, mistakes in validation, deletion rules, or billing amounts could lead to incorrect or lost records. Suggested code should be read carefully, run with fictional sample data, and checked against the requirements. The database-layer checks described in Section 8 are an example of validating behavior instead of assuming it.

9.3 Responsible Use:

AI assistance should be documented according to the work actually performed. This report does not claim specific prompts, AI-generated modules, or AI-tool validation sessions as completed activities because those details are not evidenced by the project files.

10. ALGORITHMS, COMPLEXITY AND PERFORMANCE

10.1 Appointment Scheduling Algorithm:

- Load the patient and doctor options from the database as "id - name" strings.
- Require a selected patient, a selected doctor, and a date-time value.
- Validate the date-time with datetime.strptime using the format %Y-%m-%d %H:%M.
- Extract the patient id and doctor id from the selected option text.
- Insert the appointment with the default status Scheduled using a parameterized query.
- Refresh all tables and show a confirmation message.

10.2 Safe Deletion Algorithm:

- Read the id of the selected row; warn the user if no row is selected.
- For a patient, count the appointments and bills that reference the id; for a doctor, count the appointments.
- If the count is greater than zero, show an error message and keep the record.
- Otherwise, delete the record by id.
- Refresh all tables.

10.3 Performance Considerations:

The program performs a small number of primary-key operations per action, and refresh_all() reloads every table after each change. This is simple and adequate for small data sets but grows linearly with the number of rows. Possible improvements include loading only the visible rows, adding indexes on patient_id and doctor_id, and refreshing only the affected table. No numeric benchmark is claimed because no controlled timing experiment was performed.

10.4 Data Storage Summary:

All data is stored in one SQLite file, smart_care_hospital.db. Bill amounts are stored as REAL (floating-point) values and displayed in rupees (₹). Floating-point storage can introduce small rounding differences for money, so a production billing system would normally store amounts as integer paise or use a decimal type. The application does not calculate totals, taxes, or discounts.

11. IMPLEMENTATION AND USER WORKFLOW

11.1 Installation:

- Install Python 3.10 or newer, including Tkinter.
- Extract smart_care_hospital_management.zip.
- Open the smart_care_hospital_management folder in VS Code or a terminal.
- No pip packages are required.
- Start the application using python main.py.
- The window "Smart Care - Hospital Management System" opens and smart_care_hospital.db is created automatically.

11.2 Registering Patients and Doctors:

On the Patients tab the user enters a name and age (required), selects a gender, optionally adds a phone number and address, and clicks Add patient. On the Doctors tab the user enters a name and department (required) and an optional phone number, then clicks Add doctor. A message box confirms each registration and the new record appears at the top of the table. A patient or doctor can be deleted only when no linked appointment or bill exists.

11.3 Scheduling Appointments:

On the Appointments tab the user selects a patient and a doctor from the drop-down lists, confirms or edits the date-time (pre-filled with the current time in YYYY-MM-DD HH:MM format), optionally enters a reason, and clicks Schedule. The appointment appears with the status Scheduled. Selecting it and clicking Mark selected as completed changes the status to Completed.

11.4 Billing and Payment Status:

On the Billing tab the user selects a patient, enters an amount in rupees and an optional description, and clicks Create unpaid bill. The bill is saved as Unpaid with the current date and time. Selecting a bill and clicking Mark selected as paid changes its payment status to Paid.

12. TESTING, RESULTS AND LIMITATIONS

121 Verification Results:

No unit-test file or test-results file was supplied with the project. The checks listed in Section 8 were run while preparing this report: main.py compiled without errors, and the database layer behaved as expected on a temporary database. The Tkinter interface could not be launched in the environment used, so on-screen behavior (layout, message boxes, and button actions) was reviewed from the source code and was not observed running.

122 Validation Table:

Test case	Expected behavior	Recorded status
Syntax check of main.py	File compiles without errors	Passed
Schema creation (run twice)	Four tables created; repeat run causes no error	Passed
Default statuses	Appointment = Scheduled; bill = Unpaid	Passed
Appointment JOIN query	Returns appointment with patient and doctor names	Passed
Missing required value	Patient with no name is rejected (NOT NULL)	Passed
Special characters in text	Stored as plain text; tables unaffected	Passed
GUI launch and button actions	Window opens; forms and buttons work	Not run (Tkinter unavailable)

123 Limitations:

- There is no login, user role, or audit log, and data is stored unencrypted in a local file.

- Foreign keys are declared, but SQLite does not enforce them unless PRAGMA foreign_keys is enabled; direct SQL accepted an appointment for non-existent patient and doctor ids. Through the interface, ids come from drop-down lists and deletion checks are done in code.

- The same doctor can be booked at the same date and time more than once, and past dates are accepted.

- Records cannot be edited, and appointments and bills cannot be cancelled or deleted.

- Phone numbers and addresses are not format-checked, and duplicate patients are not detected.

- Money is stored as floating-point numbers, and there is no totals or reporting view.

- All tables are refreshed after every action and database calls run on the interface thread.

- The interface was not run in the checking environment, and no automated tests, performance benchmarks, or CI/CD configuration are included.

124 Safety Recommendations:

Use only fictional data, as stated in the README. Keep a copy of smart_care_hospital.db before experimenting, and do not use the application for real patient or payment information until authentication, permission control, audit logging, secure storage, backups, and stronger validation are added.

13. CONCLUSION AND FUTURE ENHANCEMENTS

Smart Care demonstrates a practical application of Python programming and computational thinking to a hospital front-desk workflow. It combines a Tkinter graphical interface, a four-table SQLite database, input validation, JOIN-based displays, status workflows for appointments and bills, and protective deletion rules. Its single-file, dependency-free design makes it easy to run and understand, while the limitations documented in this report point to clear ways of making it more robust.

13.1 Future Enhancements:

- Add user login with role-based permissions (for example administrator, doctor, and receptionist) and an audit log.
- Enable PRAGMA foreign_keys and add indexes on the foreign-key columns.
- Prevent double-booking of a doctor and reject appointment times in the past.
- Add editing, cancellation, and search or filter options for all records.
- Replace typed date-time text with a calendar or date picker.
- Store money as integer paise or decimals, and add totals, invoices, and printable reports.
- Move database code into a separate module, and add automated unit tests with a temporary database, plus linting.
- Add backup and export of data, and protect stored personal data with encryption or secure storage.

14. REFERENCES

• Project Defence & Demonstration: Computational Thinking and Programming, supplied course document.

• Smart Care project source code: main.py and README.md.

• Python documentation: https://docs.python.org/3/

• Tkinter documentation: https://docs.python.org/3/library/tkinter.html

• SQLite documentation: https://www.sqlite.org/docs.html

• Python sqlite3 module documentation: https://docs.python.org/3/library/sqlite3.html